

Tema 3.

Preprocesamiento y análisis morfosintáctico con spaCy

Preparar un corpus sanitario para traducción automática neuronal

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Índice de contenidos

Contexto profesional.....	1
Qué vas a aprender en esta unidad.....	1
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	2
1.1. El preprocesamiento como etapa fundacional del PLN.....	2
1.2. Qué es un pipeline lingüístico.....	3
1.3. Preprocesar es modelar: reducir ruido sin destruir información.....	3
1.4. Normalización textual y limpieza de puntuación.....	4
1.5. Tokenización: segmentar no es solo dividir palabras.....	4
1.6. Stopwords: una categoría útil, pero dependiente de la tarea.....	5
1.7. spaCy y NLTK: dos herramientas complementarias.....	5
1.8. Lematización: reducir variación flexiva sin anular la gramática.....	6
1.9. Análisis morfosintáctico: de las categorías gramaticales a la terminología.....	6
1.10. Pandas para lingüistas: organizar, revisar y exportar resultados.....	6
1.11. El caso de los prospectos farmacéuticos.....	7
1.12. Preprocesamiento y traducción automática neuronal.....	8
1.13. Buenas prácticas y limitaciones del pipeline.....	8
1.14. Conclusiones del marco teórico.....	8
2. Preprocesamiento y análisis morfosintáctico con Python.....	9
2.1. Instalación y recursos lingüísticos.....	9
2.2. Cargar un texto especializado.....	9
2.3. Normalización: convertir a minúsculas.....	10
2.4. Limpieza de puntuación con expresiones regulares.....	11
2.5. Tokenización: convertir texto en unidades.....	11
2.6. Stopwords: palabras vacías y decisiones de filtrado.....	12
2.7. Stopwords en textos técnicos: el caso de la negación.....	13
2.8. Lematización: reducir variantes a una forma base.....	13
2.9. Tokenizar no es lematizar.....	14
2.10. Frecuencias con Counter.....	14
2.11. Glosario preliminar de lemas.....	15
2.12. Análisis morfosintáctico.....	15

2.13. Por qué interesan los sustantivos en textos técnicos.....	16
2.14. Frecuencia por categoría gramatical.....	17
2.15. Glosario técnico con contexto.....	17
2.16. Segmentación de oraciones con spaCy.....	17
2.17. Versiones del texto y trazabilidad.....	18
Errores frecuentes de aprendizaje.....	18
Síntesis final.....	19
Cierre narrativo.....	19
Referencias.....	19

Título narrativo

Preprocesamiento y análisis morfosintáctico de textos médico-farmacéuticos: preparar el corpus antes de analizarlo con Python

Contexto profesional

Imagina que trabajas como lingüista computacional junior en una empresa internacional que desarrolla un sistema de traducción automática especializado en documentación médica y farmacéutica. La empresa ha recopilado miles de textos multilingües: prospectos de medicamentos, informes clínicos, artículos científicos, instrucciones de administración, fichas técnicas y materiales de farmacovigilancia.

El equipo de ingeniería quiere usar esos textos para mejorar un motor de traducción automática neuronal. Sin embargo, antes de entrenar o evaluar cualquier sistema, aparece un problema básico: los textos están en bruto. Tienen mayúsculas inconsistentes, signos de puntuación variables, palabras funcionales de alta frecuencia, formas flexionadas, duplicidades, terminología especializada y segmentos que no siempre están preparados para ser procesados por una máquina.

Tu tarea consiste en diseñar un **pipeline de preprocesamiento lingüístico**. Un pipeline es una secuencia ordenada de pasos que transforma un texto bruto en una representación más limpia, homogénea y útil para el análisis automático.

En este escenario profesional, preparar bien el corpus no es una tarea secundaria. En traducción médica y farmacéutica, una mala preparación de datos puede afectar a la terminología, a la detección de negaciones, a la calidad de los glosarios y, en última instancia, a la fiabilidad del sistema.

La idea central de esta unidad es que el procesamiento automático no empieza cuando aplicamos un modelo avanzado. Empieza mucho antes: cuando decidimos cómo normalizar, cómo tokenizar, qué eliminar, qué conservar, qué lematizar y cómo extraer las unidades lingüísticas relevantes.

Qué vas a aprender en esta unidad

Al terminar este manual, deberías poder explicar y aplicar los siguientes conocimientos:

1. Qué es el preprocesamiento textual y por qué es imprescindible en PLN.
2. Qué significa construir un pipeline de procesamiento lingüístico.
3. Cómo se instala y utiliza una biblioteca lingüística como spaCy.
4. Para qué sirve NLTK en tareas de stopwords.
5. Qué es un modelo de lengua en spaCy y qué aporta `es_core_news_sm`.
6. Qué diferencia hay entre texto bruto, texto normalizado, tokens y lemas.
7. Qué implica convertir un texto a minúsculas con `.lower()`.
8. Cómo se eliminan signos de puntuación mediante expresiones regulares.
9. Qué es la tokenización y por qué no equivale a "separar por espacios" sin más.
10. Qué son las stopwords y por qué eliminarlas puede ser útil.
11. Por qué en textos técnicos no todas las stopwords deben eliminarse automáticamente.
12. Qué es la lematización y por qué ayuda a agrupar variantes morfológicas.
13. Cómo se cuentan frecuencias con `Counter`.
14. Qué es el etiquetado morfosintáctico o PoS tagging.
15. Cómo se usan categorías como NOUN, VERB y ADJ.
16. Por qué los sustantivos suelen ser relevantes para glosarios técnicos.

17. Cómo se construye un glosario terminológico con contexto.
18. Qué decisiones lingüísticas deben documentarse en un pipeline profesional.

Mapa conceptual de la unidad

Concepto	En Python	En PLN aplicado
Pipeline	Secuencia de instrucciones	Flujo ordenado de transformación textual
spaCy	Biblioteca de PLN	Tokenización, lematización y análisis morfosintáctico
NLTK	Biblioteca clásica de PLN	Listas de stopwords y recursos lingüísticos
Modelo es_core_news_sm	Modelo preentrenado para español	Analiza texto español con reglas y estadísticas aprendidas
Normalización	.lower() y otras transformaciones	Unifica variantes superficiales
Limpieza	re.sub()	Elimina ruido como puntuación no necesaria
Tokenización	nlp(texto) y token.text	Divide el texto en unidades analizables
Stopwords	stopwords.words('spanish')	Filtra palabras funcionales frecuentes
Lematización	token.lemma_	Reduce formas flexionadas a una forma base
Frecuencia	Counter	Cuenta términos, lemas o categorías
PoS	token.pos_	Etiqueta la categoría gramatical
Glosario técnico	Sustantivos + contexto	Extrae unidades terminológicas relevantes

1. Marco teórico

El marco teórico de esta unidad debe leerse como una secuencia: primero se entiende por qué el preprocesamiento es una fase decisiva del PLN; después se estudia cómo se ordena esa fase en un pipeline; a continuación se analizan las operaciones lingüísticas principales; y, finalmente, se sitúan esas decisiones en un contexto profesional médico-farmacéutico, con herramientas concretas como spaCy, NLTK (Natural Language Toolkit, 2026) y pandas (The pandas development team, 2026a, 2026b, 2026c). El hilo conductor es que preparar un corpus no es una tarea secundaria, sino una forma de modelar lingüísticamente qué información verá después la máquina.

1.1. El preprocesamiento como etapa fundacional del PLN

El preprocesamiento textual es el conjunto de operaciones que transforma textos brutos en representaciones más regulares, comparables y explotables por un sistema computacional. Un texto real puede contener mayúsculas inconsistentes, puntuación variable, saltos de línea, abreviaturas, cifras, unidades de medida, formas flexionadas, palabras funcionales de alta frecuencia y terminología

especializada. Antes de contar, clasificar, traducir o extraer información, es necesario decidir cómo se tratarán todos esos elementos.

En PLN, la calidad de los resultados posteriores depende en parte de la calidad de la representación inicial. Por eso preprocesar no significa «limpiar por limpiar», sino preparar el texto para una tarea concreta. Una representación adecuada para contar frecuencias no tiene por qué ser adecuada para segmentar oraciones, extraer terminología o alimentar un sistema de traducción automática neuronal. La misma operación puede ser útil en una tarea y perjudicial en otra.

Esta idea es especialmente importante para alumnado de lingüística. El preprocesamiento no es solo una operación técnica: es una decisión lingüística. Cada vez que se normaliza una forma, se elimina una palabra o se reduce una variante a su lema, se está decidiendo qué rasgos del texto se consideran relevantes y cuáles se consideran prescindibles.

1.2. Qué es un pipeline lingüístico

Un pipeline lingüístico es una cadena ordenada de pasos en la que cada operación recibe una entrada, la transforma y produce una salida que será utilizada por el paso siguiente. En una unidad sobre preprocesamiento y análisis morfosintáctico, un pipeline básico puede incluir carga del texto, normalización, limpieza de puntuación, tokenización, filtrado de stopwords, lematización, recuento de frecuencias, etiquetado gramatical, extracción de sustantivos y construcción de un glosario preliminar.

El orden del pipeline importa. Si se elimina toda la puntuación antes de segmentar oraciones, pueden perderse marcas que ayudan a detectar límites oracionales. Si se eliminan stopwords antes de revisar negaciones, puede desaparecer información semántica crítica. Si se lematiza sin conservar el texto original, puede ser difícil volver al contexto real de uso.

Por eso una buena práctica consiste en conservar versiones intermedias del texto. En lugar de sobrescribir siempre la misma variable, conviene crear variables distintas para cada fase: texto original, texto normalizado, texto sin puntuación, documento procesado con spaCy y documento con puntuación conservada para segmentar oraciones. Esta trazabilidad permite comprobar errores, justificar decisiones y reproducir el análisis.

Fase	Pregunta lingüística	Riesgo si se aplica sin criterio
Normalización	¿Qué variantes formales quiero unificar?	Borrar siglas, nombres propios o marcas funcionales
Limpieza	¿Qué signos dificultan la tarea y cuáles informan?	Eliminar unidades, dosis, límites oracionales o contexto
Stopwords	¿Qué palabras frecuentes son realmente prescindibles?	Eliminar negación, temporalidad o restricción
Lematización	¿Qué variantes deben agruparse bajo una forma base?	Perder información morfológica útil
Extracción gramatical	¿Qué categorías interesan para la tarea?	Confundir núcleos terminológicos con palabras aisladas

1.3. Preprocesar es modelar: reducir ruido sin destruir información

Una de las ideas centrales de la unidad es que preprocesar equivale a modelar. No existe una limpieza universalmente correcta. Lo que suele llamarse «ruido» depende de la tarea, del dominio y de la pregunta de análisis. En recuperación de información puede interesar eliminar muchas palabras

frecuentes; en textos médicos, en cambio, algunas palabras frecuentes pueden indicar contraindicación, temporalidad o restricción.

En un corpus médico-farmacéutico, elementos aparentemente pequeños pueden ser decisivos. La palabra «no» cambia el sentido de «se recomienda su uso» frente a «no se recomienda su uso». La forma «sin» puede introducir una condición clínica. Una cifra seguida de una unidad, como «10 mg/kg», no es un detalle tipográfico, sino información especializada. Por tanto, el pipeline ideal no es el más agresivo, sino el más adecuado al objetivo.

La literatura sobre adaptación de dominio en traducción neuronal y sobre procesamiento biomédico insiste en que los datos especializados requieren decisiones de representación propias. Un sistema que funciona aceptablemente en textos generales puede degradarse cuando se enfrenta a terminología médica, abreviaturas, negación clínica o unidades de medida. La preparación del corpus forma parte de esa adaptación de dominio, no solo de una limpieza previa (Chu y Wang, 2018; Neumann *et al.*, 2019).

1.4. Normalización textual y limpieza de puntuación

La normalización textual agrupa operaciones destinadas a reducir variación superficial: pasar a minúsculas, unificar caracteres, tratar puntuación, resolver espacios o estandarizar formas equivalentes. En el cuaderno se trabaja de forma inicial con `lower`, que convierte el texto a minúsculas y permite contar de forma conjunta variantes como «Paciente» y «paciente».

Esta operación puede ser útil para análisis de frecuencias, pero tiene límites. En textos especializados, las mayúsculas pueden distinguir siglas como AINE, nombres comerciales, códigos, instituciones o entidades clínicas. Convertir todo a minúsculas puede ser razonable para un glosario exploratorio, pero no necesariamente para una versión final del corpus.

La limpieza de puntuación mediante expresiones regulares plantea una tensión parecida. Eliminar signos puede facilitar ciertos recuentos, pero también puede destruir información sintáctica o semántica. Los puntos ayudan a segmentar oraciones; las comas pueden delimitar incisos; los guiones pueden formar parte de términos compuestos; los símbolos y porcentajes pueden ser relevantes en textos médicos. La pregunta profesional no es «¿puedo eliminar esto?», sino «¿qué pierdo si lo elimino?».

1.5. Tokenización: segmentar no es solo dividir palabras

La tokenización es el proceso por el que un flujo de caracteres se convierte en unidades manipulables por el sistema. Suele explicarse como dividir un texto en palabras, pero esa explicación se queda corta. Un token puede ser una palabra, un número, un signo de puntuación, una abreviatura, una unidad especial o una subunidad, según el tokenizador y la tarea.

En spaCy, al procesar un texto con `nlp(texto)`, se obtiene un objeto `Doc` que contiene objetos `Token`. Cada token conserva atributos como la forma textual, el lema, la categoría gramatical y, en su caso, información adicional. Esta estructura permite que los pasos posteriores del pipeline no trabajen solo con cadenas, sino con unidades lingüísticamente anotadas.

En traducción automática neuronal, la tokenización ha sido especialmente importante por el uso de subpalabras. Métodos como BPE o SentencePiece ayudan a tratar palabras raras y morfología productiva, pero también muestran que tokenizar no es una operación trivial. La forma en que se segmenta el texto condiciona qué regularidades puede aprender el sistema (Kudo y Richardson, 2018; Schmidt *et al.*, 2024; Sennrich *et al.*, 2016).

1.6. Stopwords: una categoría útil, pero dependiente de la tarea

Las stopwords o palabras vacías son unidades de alta frecuencia que, en algunas tareas, aportan poca información temática o discriminativa. En español suelen incluir artículos, preposiciones, conjunciones, pronombres y formas funcionales. Su eliminación ha sido clásica en recuperación de información, clasificación textual, análisis de frecuencias y representación vectorial basada en conteos (Manning *et al.*, 2008).

Sin embargo, no existe una lista de stopwords universal. La utilidad de eliminar una palabra depende del género textual, del dominio, de la lengua y de la tarea. En textos médicos, palabras como «no», «sin», «antes», «después», «durante», «excepto» o «contra» pueden codificar negación, temporalidad, condición o contraindicación. Eliminarlas de manera automática puede alterar el significado del texto.

Por eso, en corpus especializados conviene entender la lista de stopwords como un punto de partida revisable. El criterio profesional puede exigir conservar negaciones, ajustar la lista general o construir una lista personalizada. En esta unidad, el caso de la negación funciona como advertencia metodológica: una palabra muy frecuente no siempre es irrelevante.

1.7. spaCy y NLTK: dos herramientas complementarias

spaCy y NLTK permiten trabajar con PLN en Python, pero responden a orientaciones distintas. NLTK es una biblioteca clásica, muy valiosa en docencia y en trabajo introductorio con corpus, porque permite ver de forma explícita recursos y operaciones: cargar una lista de stopwords, inspeccionarla, convertirla en conjunto y filtrarla. Su ventaja pedagógica es la transparencia del proceso (Bird *et al.*, 2009; Loper y Bird, 2002).

spaCy, en cambio, está orientada a pipelines lingüísticos eficientes e integrados. Un modelo de spaCy para español no funciona solo como diccionario: permite tokenizar, lematizar, etiquetar morfosintácticamente, segmentar oraciones y consultar atributos lingüísticos de cada token. En spaCy, las stopwords pueden consultarse como parte de los datos del modelo o mediante el atributo `token.is_stop`.

Las listas de stopwords de NLTK y spaCy no tienen por qué coincidir. Cada biblioteca mantiene recursos propios y criterios de inclusión distintos. Por eso, dos procedimientos que parecen equivalentes pueden producir salidas diferentes: conservarán o eliminarán conjuntos de tokens no idénticos. Esta diferencia no debe interpretarse como que una herramienta sea siempre mejor que la otra, sino como una decisión metodológica que debe documentarse.

Para una práctica inicial, NLTK ayuda a comprender el filtrado de forma explícita. Para un pipeline completo, spaCy resulta más natural porque integra el filtrado con tokenización, lematización, PoS tagging y segmentación. En proyectos profesionales, lo más riguroso es elegir la herramienta según la tarea, revisar sus efectos y justificar la decisión.

Para saber más, se recomienda leer el artículo complementario «Diferencias entre las stopwords de spaCy y NLTK en el procesamiento textual».

1.8. Lematización: reducir variación flexiva sin anular la gramática

La lematización asocia una forma flexionada con su forma canónica o lema. En español, esta operación permite agrupar formas como «paciente» y «pacientes», o relacionar ciertas formas verbales con su infinitivo. Para análisis de frecuencias y glosarios preliminares, lematizar puede reducir dispersión morfológica y facilitar la lectura de patrones.

Lematizar no equivale a tokenizar. Tokenizar responde a la pregunta «¿cuáles son las unidades del texto?». Lematizar responde a otra: «¿cuál es la forma de referencia de cada unidad?». Ambas operaciones son complementarias y ocupan lugares distintos en el pipeline. Primero se necesitan unidades; después puede decidirse si conviene reducirlas a lemas.

También conviene evitar una interpretación excesivamente automática. En dominios especializados, los modelos pueden producir lemas dudosos, sobre todo con abreviaturas, tecnicismos, formas raras o usos fuera de dominio. La lematización moderna se beneficia del contexto, pero sus resultados deben revisarse cuando el objetivo sea construir glosarios o preparar recursos terminológicos (Toporkov y Agerri, 2024).

1.9. Análisis morfosintáctico: de las categorías gramaticales a la terminología

El análisis morfosintáctico añade a cada token información gramatical. Una de sus tareas principales es el etiquetado PoS, que asigna categorías como sustantivo, verbo, adjetivo, adverbio, determinante o preposición. Esta capa permite pasar de una simple lista de palabras a una representación más estructurada del texto.

En textos médicos y farmacéuticos, los sustantivos son especialmente relevantes porque muchos candidatos terminológicos son nombres o grupos nominales: «ibuprofeno», «tratamiento», «efectos adversos», «úlcera gástrica», «principio activo», «insuficiencia hepática» o «sistema nacional de farmacovigilancia». Extraer sustantivos individuales puede ser un primer paso, pero no basta para capturar toda la terminología.

De sustantivos a terminología hay un salto metodológico. La terminología especializada suele estar formada por unidades multipalabra, con patrones como sustantivo + adjetivo, sustantivo + preposición + sustantivo o grupos nominales más extensos. Por eso, la categoría gramatical no debe verse solo como etiqueta descriptiva: puede convertirse en una regla de búsqueda para localizar candidatos terminológicos.

El marco de Universal Dependencies y los tagsets universales ayudan a entender que estas etiquetas no son arbitrarias: responden a tradiciones de anotación lingüística formalizadas y comparables entre lenguas (de Marneffe *et al.*, 2021; Petrov *et al.*, 2012).

1.10. Pandas para lingüistas: organizar, revisar y exportar resultados

spaCy y NLTK producen o facilitan análisis lingüísticos, pero muchas veces los resultados necesitan organizarse en una estructura consultable. Ahí aparece pandas. Pandas no analiza lingüísticamente el texto; su función es convertir resultados en datos tabulares: filas y columnas que pueden inspeccionarse, filtrarse, ordenarse, calcularse y exportarse.

La estructura central de pandas es el DataFrame. Para un perfil lingüístico, puede entenderse como una tabla de anotación: cada fila representa una unidad —un token, una oración, un documento, un lema o un candidato terminológico— y cada columna almacena una variable —forma textual, lema,

categoría gramatical, frecuencia, documento de origen o contexto—. Esta forma de organización conecta con el principio de *tidy data*: cada variable en una columna, cada observación en una fila y cada tipo de unidad en una tabla (Wickham, 2014).

En PLN aplicado, pandas aporta claridad y reproducibilidad. Permite revisar si los lemas son correctos, filtrar solo sustantivos, calcular frecuencias por categoría gramatical, ordenar resultados y exportar a CSV. El CSV es un archivo de texto plano que representa datos tabulares y puede abrirse en Excel, LibreOffice, Google Sheets, R o Python. Exportar resultados permite compartirlos, revisarlos fuera del cuaderno y conservar evidencias del análisis.

La distinción metodológica es importante: spaCy analiza; NLTK proporciona recursos; pandas organiza y exporta. En una práctica profesional, esta separación ayuda a documentar qué parte del código realiza análisis lingüístico y qué parte convierte ese análisis en un producto revisable.

Estructura	Qué representa	Utilidad en PLN
Lista	Elementos en orden	Guardar tokens o lemas de forma simple
Diccionario	Pares clave-valor	Describir una unidad con atributos
Lista de diccionarios	Muchas unidades anotadas	Preparar datos para una tabla
DataFrame	Tabla con filas y columnas	Revisar, filtrar, contar y exportar resultados
CSV	Archivo tabular en texto plano	Conservar y compartir los resultados

Para saber más, se recomienda leer el artículo complementario «Pandas para lingüistas».

1.11. El caso de los prospectos farmacéuticos

Los prospectos farmacéuticos son un buen escenario para esta unidad porque combinan instrucciones, advertencias, contraindicaciones, dosis, unidades, condiciones temporales, síntomas y terminología médica. Además, son textos en los que pequeños cambios pueden alterar el significado: una negación, una cifra, una unidad o una preposición pueden tener impacto interpretativo.

Un pipeline mal diseñado podría eliminar información crítica. Si elimina «no», cambia la polaridad de una recomendación. Si elimina puntuación indiscriminadamente, puede dificultar la segmentación de instrucciones y advertencias. Si no lematiza, puede dispersar formas relacionadas. Si extrae palabras sin contexto, puede producir un glosario pobre o ambiguo.

Por eso, en este dominio el preprocesamiento debe responder al texto y a la finalidad. No se trata de aplicar una receta general, sino de decidir qué transformaciones ayudan a preparar el corpus sin comprometer información clínica, terminológica o traductológica relevante. Recursos como PharmaCoNER o NUBes muestran que el español biomédico requiere atención específica a entidades farmacológicas, negación e incertidumbre (González-Agirre *et al.*, 2019; Lima López *et al.*, 2020).

1.12. Preprocesamiento y traducción automática neuronal

La traducción automática neuronal aprende patrones a partir de grandes cantidades de datos. Aunque los modelos actuales son más potentes que los enfoques clásicos, siguen siendo sensibles al dominio, a la cobertura terminológica y a la forma en que el texto se representa. Un corpus especializado mal preparado puede introducir ruido, dispersar términos o hacer que el sistema aprenda equivalencias poco fiables.

Un buen preprocesamiento puede ayudar a reducir variación superficial innecesaria, detectar términos frecuentes, preparar glosarios, localizar segmentos problemáticos y documentar decisiones. Pero también puede perjudicar si elimina demasiada información. Para traducción médica, por ejemplo, no siempre conviene alimentar un sistema final con texto excesivamente simplificado: puede ser útil preprocesar para analizar y depurar, pero conservar información rica para traducir.

La relación entre preprocesamiento y traducción automática debe entenderse así: el pipeline no sustituye al modelo, pero condiciona los datos que el modelo aprenderá o que el equipo humano revisará. En ámbitos de alto riesgo, la terminología no es un detalle estilístico, sino una condición de seguridad y adecuación (Chu y Wang, 2018; Semenov *et al.*, 2025).

1.13. Buenas prácticas y limitaciones del pipeline

Un pipeline profesional debe ser reproducible, justificable y revisable. Esto implica conservar siempre una copia del texto original, crear variables para cada versión procesada, revisar muestras de salida, documentar la lista de stopwords utilizada, comprobar cómo se tratan siglas, cifras y unidades, y no eliminar negaciones sin una razón explícita.

También conviene reconocer las limitaciones de las herramientas. spaCy puede equivocarse al tokenizar, lematizar o etiquetar textos especializados. NLTK ofrece recursos útiles, pero sus listas generales no están diseñadas específicamente para corpus biomédicos. pandas organiza resultados, pero no decide si una etiqueta o un lema son lingüísticamente correctos.

Estas limitaciones no invalidan el uso de las herramientas; simplemente las sitúan en su lugar. El objetivo de la unidad no es confiar ciegamente en spaCy, NLTK o pandas, sino aprender a utilizarlos con criterio lingüístico: observar resultados, detectar errores, adaptar decisiones y justificar el pipeline.

1.14. Conclusiones del marco teórico

Antes de modelar el lenguaje, hay que decidir cómo representarlo. El preprocesamiento convierte un texto complejo en una estructura computable, pero esa conversión nunca es neutra. Normalizar, tokenizar, eliminar stopwords, lematizar, etiquetar categorías gramaticales, extraer sustantivos o exportar resultados son decisiones técnicas con consecuencias lingüísticas.

La secuencia conceptual de la unidad puede resumirse así: el texto bruto se convierte en versiones procesadas; esas versiones permiten obtener tokens, lemas, categorías y frecuencias; esos resultados pueden organizarse en tablas; y todo el proceso debe documentarse para que sea revisable. En corpus médico-farmacéuticos, esta cadena exige especial cuidado porque la terminología, la negación, la puntuación y las unidades pueden ser semánticamente críticas.

Por tanto, el objetivo formativo no es solo aprender a ejecutar código con spaCy, NLTK o pandas. El objetivo es aprender a diseñar, justificar y evaluar un pipeline lingüístico: una forma ordenada de transformar textos en datos sin perder de vista que esos datos siguen siendo lenguaje.

2. Preprocesamiento y análisis morfosintáctico con Python

2.1. Instalación y recursos lingüísticos

Para trabajar con estas herramientas, es necesario instalar bibliotecas y descargar recursos. En un entorno como Google Colab puede hacerse con comandos de instalación:

```
!pip install --quiet spacy nltk
!python -m nltk.downloader stopwords
!python -m spacy download es_core_news_sm
```

Estos comandos cumplen tres funciones:

- Instalan las bibliotecas `spacy` y `nltk`.
- Descargan la lista de stopwords de NLTK.
- Descargan el modelo pequeño de español de spaCy.

Después, en Python se importan las bibliotecas necesarias:

```
import spacy
import nltk
from nltk.corpus import stopwords
import re
from collections import Counter
```

Cada importación responde a una necesidad:

Importación	Para qué sirve
<code>spacy</code>	Procesar texto con modelos lingüísticos
<code>nltk</code>	Descargar y gestionar recursos
<code>stopwords</code>	Acceder a listas de palabras vacías
<code>re</code>	Usar expresiones regulares
<code>Counter</code>	Contar frecuencias de elementos

El siguiente paso es cargar el modelo:

```
nlp = spacy.load("es_core_news_sm")
```

`nlp` funciona como una herramienta de análisis. Cuando se aplica a un texto, devuelve un objeto `Doc` con información lingüística.

2.2. Cargar un texto especializado

Un corpus puede empezar como un fragmento de texto guardado en una variable:

```
texto = """
La administración de ibuprofeno debe realizarse con precaución en pacientes con
antecedentes de úlcera gástrica.
Los AINE pueden provocar efectos adversos, especialmente en pacientes de edad avanzada.
Se recomienda consultar con un profesional de la salud antes de iniciar cualquier
tratamiento.
"""
```

Este texto pertenece al ámbito médico-farmacéutico. Contiene términos técnicos como:

- administración
- ibuprofeno
- pacientes
- antecedentes
- úlcera gástrica
- AINE
- efectos adversos
- tratamiento

Desde el punto de vista lingüístico, estos términos no deben perderse. Son precisamente las unidades que pueden formar parte de un glosario técnico o de un sistema de traducción especializada.

La carga del texto es el punto de partida. Todo lo que hagamos después transformará esta materia prima.

2.3. Normalización: convertir a minúsculas

La normalización busca reducir variaciones superficiales. Una de las operaciones más comunes es convertir todo el texto a minúsculas:

```
texto_normalizado = texto.lower()
```

El método `.lower()` devuelve una nueva cadena en la que las letras aparecen en minúscula. Esto permite tratar como equivalentes formas que solo difieren por mayúscula inicial:

Forma original	Forma normalizada
Paciente	paciente
paciente	paciente
TRATAMIENTO	tratamiento

Esta operación es útil para contar frecuencias. Si no normalizamos, `Paciente` y `paciente` podrían contarse como formas distintas.

Pero en textos técnicos hay que ser prudentes. Algunas mayúsculas contienen información:

- Siglas: AINE, VIH, ADN.
- Nombres comerciales.
- Unidades o códigos.
- Nombres propios de instituciones.

Convertir todo a minúsculas puede ser adecuado para un glosario preliminar, pero no siempre para una aplicación final. La decisión depende de la tarea.

2.4. Limpieza de puntuación con expresiones regulares

Una forma habitual de eliminar signos de puntuación es usar `re.sub()`:

```
texto_sin_puntuacion = re.sub(r'^\w\s', '', texto_normalizado)
```

Esta línea sustituye por una cadena vacía todo lo que coincida con el patrón.

El patrón es:

```
r'^\w\s'
```

Su significado puede desglosarse así:

Parte	Significado
<code>r"</code>	Cadena raw
<code>[...]</code>	Clase de caracteres
<code>^</code> dentro de corchetes	Negación: "todo lo que no sea..."
<code>\w</code>	Caracteres de palabra
<code>\s</code>	Espacios en blanco

Por tanto, el patrón significa: busca todo carácter que no sea carácter de palabra ni espacio. En la práctica, elimina muchos signos de puntuación.

Esta limpieza puede facilitar recuentos y tokenización, pero también puede eliminar información útil:

- Puntos que marcan final de oración.
- Comas que indican estructura sintáctica.
- Guiones en términos compuestos.
- Porcentajes o símbolos en textos médicos.

Ejemplo:

```
10 mg/kg
```

Si se eliminan símbolos sin cuidado, puede perderse la relación entre cantidad y unidad. En textos biomédicos, la puntuación y los símbolos pueden ser semánticamente importantes.

2.5. Tokenización: convertir texto en unidades

La tokenización consiste en dividir un texto en unidades mínimas de análisis llamadas tokens. Un token puede ser una palabra, un número, un signo de puntuación o una unidad especial, según el tokenizador.

Con `spaCy`, se procesa el texto así:

```
doc = nlp(texto_sin_puntuacion)
tokens = [token.text for token in doc]
```

`doc` es un objeto de `spaCy` que contiene el análisis lingüístico. Al recorrer `doc`, obtenemos objetos `Token`. Cada token tiene atributos.

El atributo `token.text` devuelve la forma textual del token:

```
tokens = [token.text for token in doc]
```

La tokenización es más que separar por espacios. Un tokenizador lingüístico puede reconocer signos, abreviaturas, contracciones, números y límites de una forma más sofisticada que un simple `.split()`.

En un pipeline profesional, tokenizar correctamente es esencial porque muchos pasos posteriores dependen de los tokens:

- Eliminación de stopwords.
- Lematización.
- Conteo de frecuencias.
- Etiquetado gramatical.
- Extracción terminológica.

Si la tokenización es mala, los pasos siguientes heredarán ese error.

2.6. Stopwords: palabras vacías y decisiones de filtrado

Las stopwords son palabras muy frecuentes que suelen aportar poca información léxica en ciertas tareas. En español, pueden incluir artículos, preposiciones, conjunciones y algunos pronombres.

Con NLTK se obtiene la lista así:

```
stopwords_es = stopwords.words('spanish')
```

Después se filtran los tokens:

```
tokens_filtrados = [token for token in tokens if token not in stopwords_es]
```

La lectura es:

```
Conserva cada token si no está en la lista de stopwords en español.
```

Esta operación puede ser útil si queremos quedarnos con palabras de mayor carga léxica, como sustantivos, verbos o adjetivos relevantes.

Por ejemplo, en un texto farmacéutico puede interesar conservar:

```
ibuprofeno, úlcera, gástrica, efectos, adversos, tratamiento
```

y eliminar palabras como:

```
la, de, con, en, los
```

Pero esta decisión no es automática ni universal. En algunos contextos, palabras aparentemente funcionales son críticas.

2.7. Stopwords en textos técnicos: el caso de la negación

En textos médicos, algunas palabras que podrían considerarse funcionales tienen un valor semántico decisivo. La negación es el caso más claro.

Compara:

```
Se recomienda su uso.  
No se recomienda su uso.
```

La diferencia entre ambas frases depende de `no`. Si una lista de stopwords elimina `no`, el sistema puede perder una parte esencial del significado.

También son delicadas formas como:

- sin
- antes
- después
- durante
- excepto
- contra

En textos farmacéuticos, estas palabras pueden indicar condiciones, restricciones, secuencias temporales o contraindicaciones.

Por tanto, en contextos técnicos conviene revisar y adaptar la lista de stopwords. Una opción profesional sería crear una lista personalizada:

```
stopwords_personalizadas = [w for w in stopwords_es if w not in ["no", "sin", "antes"]]
```

Esta decisión muestra que el preprocesamiento no es solo técnico. Es lingüístico y profesional.

2.8. Lematización: reducir variantes a una forma base

La lematización consiste en obtener la forma canónica o de diccionario de una palabra. Esa forma se llama lema.

Ejemplos:

Forma flexionada	Lema
pacientes	paciente
recomienda	recomendar
provocar	provocar
adversos	adverso
tratamientos	tratamiento

Con spaCy, el lema se obtiene mediante:

```
token.lemma_
```

Una lista de lemas filtrados puede construirse así:

```
lemmas = [token.lemma_ for token in doc if token.text not in stopwords_es]
```

La lematización es muy útil para análisis de frecuencia. Sin lematizar, paciente y pacientes se contarían como formas diferentes. Con lematización, pueden agruparse bajo una misma forma base.

Esto ayuda a construir glosarios más coherentes y a reducir dispersión morfológica en los datos.

2.9. Tokenizar no es lematizar

Es importante no confundir tokenización y lematización.

Proceso	Qué hace	Resultado
Tokenización	Divide el texto en unidades	pacientes, tratamientos, adversos
Lematización	Reduce cada unidad a su forma base	paciente, tratamiento, adverso

Tokenizar responde a la pregunta:

¿Cuáles son las unidades del texto?

Lematizar responde a otra:

¿Cuál es la forma canónica de cada unidad?

Ambas operaciones son complementarias. Primero necesitamos unidades; después podemos obtener sus lemas.

En traducción automática y análisis terminológico, lematizar puede ayudar a detectar que varias formas pertenecen a la misma familia léxica o al mismo término base.

2.10. Frecuencias con Counter

Una vez obtenidos tokens o lemas, suele interesar contar cuántas veces aparece cada uno. Python ofrece Counter en el módulo collections:

```
from collections import Counter
frecuencias = Counter(lemmas)
```

Counter crea una estructura parecida a un diccionario:

```
{
  "paciente": 2,
  "tratamiento": 1,
  "salud": 1
}
```

Después se puede recorrer ordenando alfabéticamente:

```
for palabra in sorted(frecuencias):
    print(f"- {palabra} ({frecuencias[palabra]})")
```

O por frecuencia:

```
frecuencias.most_common()
```

El recuento de frecuencias permite identificar términos recurrentes. En un corpus técnico, las palabras frecuentes pueden apuntar a conceptos centrales del dominio.

2.11. Glosario preliminar de lemas

Un glosario preliminar de lemas es una lista de formas canónicas con sus frecuencias. No es todavía un glosario terminológico validado, pero sirve como primera aproximación.

Ejemplo de salida:

```
- adverso (1)
- antecedente (1)
- ibuprofeno (1)
- paciente (2)
- tratamiento (1)
```

Este glosario ayuda a ver qué contenido léxico permanece después del filtrado. También permite detectar problemas:

- Lemas incorrectos.
- Términos partidos en varias unidades.
- Palabras técnicas eliminadas por error.
- Formas que deberían agruparse y no se han agrupado.

En un proyecto real, este glosario debería revisarse por una persona con conocimiento lingüístico y terminológico.

2.12. Análisis morfosintáctico

El análisis morfosintáctico consiste en asignar información gramatical a las palabras de un texto. Una de sus tareas más habituales es el etiquetado PoS, del inglés *Part-of-Speech tagging*.

El etiquetado PoS asigna categorías como:

Etiqueta	Categoría
NOUN	Sustantivo común
PROPN	Nombre propio
VERB	Verbo
ADJ	Adjetivo
ADV	Adverbio
ADP	Adposición o preposición
DET	Determinante
CCONJ	Conjunción coordinante

Con spaCy se accede a la categoría mediante:

```
token.pos_
```

Un recorrido básico sería:

```
for token in doc:  
    print(token.text, token.pos_)
```

Este análisis permite ir más allá de la forma superficial. Ya no solo vemos qué palabra aparece, sino qué papel gramatical general tiene.

2.13. Por qué interesan los sustantivos en textos técnicos

En textos médicos y farmacéuticos, muchos términos importantes son sustantivos o grupos nominales:

- administración
- ibuprofeno
- pacientes
- antecedentes
- úlcera
- efectos
- tratamiento
- sobredosis
- farmacovigilancia

Por eso, una estrategia inicial para crear un glosario técnico consiste en extraer sustantivos:

```
sustantivos = [token.text.lower() for token in doc if token.pos_ == "NOUN"]
```

Esta línea conserva solo tokens cuya categoría gramatical sea NOUN.

Después se cuentan:

```
frecuencia_sustantivos = Counter(sustantivos)
```

Este procedimiento no captura toda la terminología, porque muchos términos técnicos son unidades multipalabra:

```
úlcera gástrica  
efectos adversos  
principio activo  
insuficiencia hepática  
centro médico
```

Aun así, la extracción de sustantivos es un primer paso útil para localizar núcleos terminológicos.

2.14. Frecuencia por categoría gramatical

También se puede contar cuántas veces aparece cada categoría gramatical:

```
categorias = [token.pos_ for token in doc]
conteo_categorias = Counter(categorias)
```

Después se pueden mostrar solo algunas categorías:

```
for categoria, freq in conteo_categorias.items():
    if categoria in ["NOUN", "VERB", "ADJ"]:
        print(f"{categoria}: {freq}")
```

Este análisis da una visión general del tipo de texto. Un texto técnico suele tener una densidad importante de sustantivos y adjetivos especializados. Un texto instructivo puede tener muchos verbos en formas imperativas o perífrasis de obligación.

La distribución gramatical no es una interpretación completa, pero ayuda a caracterizar el corpus.

2.15. Glosario técnico con contexto

Un glosario útil no debería limitarse a listar palabras. Debe mostrar contexto. El contexto permite saber cómo se usa un término y si realmente es relevante.

Una estrategia consiste en recorrer las oraciones y mostrar cada sustantivo en su segmento:

```
for sent in doc.sents:
    for palabra in frecuencia_sustantivos:
        if palabra in sent.text.lower():
            print(f"- [{palabra}] → {sent.text.strip()}")
```

La salida puede tener esta forma:

```
- [ibuprofeno] → La administración de ibuprofeno debe realizarse con precaución...
- [pacientes] → Los AINE pueden provocar efectos adversos...
```

El contexto es fundamental para terminología. Una palabra aislada puede ser ambigua. En su oración, se ve su función, sus colocaciones y su relación con otros términos.

En traducción especializada, un glosario con contexto es mucho más útil que una lista descontextualizada.

2.16. Segmentación de oraciones con spaCy

spaCy permite acceder a las oraciones detectadas mediante:

```
doc.sents
```

Esto es útil para construir glosarios con contexto o para generar informes por oración.

Sin embargo, la segmentación depende de la puntuación y del modelo. Si antes eliminamos toda la puntuación, el sistema puede tener más dificultades para detectar límites oracionales. Por eso, para glosarios con contexto puede convenir procesar una versión que conserve la puntuación:

```
doc_con_puntuacion = nlp(texto_normalizado)
```

Esta decisión muestra de nuevo la importancia de conservar distintas versiones del texto. Una versión sin puntuación puede servir para contar palabras; una versión con puntuación puede servir para segmentar oraciones.

2.17. Versiones del texto y trazabilidad

Un pipeline responsable conserva versiones intermedias del texto:

Variable	Qué contiene	Para qué sirve
texto	Texto original	Referencia y comparación
texto_normalizado	Texto en minúsculas	Unificar formas
texto_sin_puntuacion	Texto sin signos	Recuento y filtrado básico
doc	Texto procesado por spaCy	Tokens, lemas, PoS
doc_con_puntuacion	Texto con puntuación procesado	Oraciones y contexto

Esta trazabilidad es muy importante en trabajos profesionales. Si un resultado parece extraño, podemos volver atrás y comprobar en qué paso se produjo el problema.

También ayuda a documentar decisiones. Un cliente o un equipo técnico puede preguntar por qué se eliminó la puntuación, por qué se conservaron ciertas palabras o por qué se extrajeron solo sustantivos. La respuesta debe poder reconstruirse.

Errores frecuentes de aprendizaje

Pensar que limpiar siempre mejora el texto: Limpiar demasiado puede eliminar información crucial.

Confundir token con palabra ortográfica: Un token depende del tokenizador. No siempre coincide con la intuición escolar de «palabra».

Eliminar stopwords sin revisar la lista: En textos técnicos, algunas stopwords pueden ser semánticamente importantes.

Creer que lematizar es traducir: Lematizar obtiene una forma base dentro de la misma lengua. No produce equivalencias entre lenguas.

Usar el texto sin puntuación para todo: Puede servir para algunas tareas, pero perjudicar la segmentación en oraciones.

No revisar los errores del modelo: spaCy ofrece análisis automático, pero conviene revisar muestras, especialmente en dominios especializados.

Extraer solo palabras aisladas: La terminología técnica suele estar formada por grupos de palabras.

Síntesis final

El preprocesamiento es una fase decisiva en cualquier proyecto de PLN. En un corpus médico-farmacéutico, no basta con dividir el texto en palabras. Hay que decidir cómo normalizar, qué limpiar, qué conservar, qué filtrar y qué representar como unidad relevante.

spaCy permite procesar texto español y obtener tokens, lemas, categorías gramaticales y oraciones. NLTK aporta recursos como listas de stopwords. `re` permite limpiar mediante patrones. `Counter` ayuda a contar frecuencias. Juntas, estas herramientas permiten construir un pipeline inicial de análisis lingüístico.

Pero la herramienta más importante sigue siendo el criterio lingüístico. Un pipeline profesional no aplica reglas de forma ciega. Las adapta al dominio, revisa sus efectos y documenta sus decisiones.

En el escenario de la empresa de traducción automática biomédica, el objetivo no es producir una lista bonita de palabras. El objetivo es preparar datos fiables para que los sistemas posteriores trabajen con menos ruido y con más información útil.

Cierre narrativo

La empresa internacional no necesita únicamente más textos. Necesita textos preparados con criterio. En traducción automática biomédica, el corpus no es una materia prima neutra: contiene decisiones, riesgos y conocimiento especializado.

Tu papel como lingüista computacional consiste en transformar textos brutos en datos lingüísticamente útiles. Para ello combinas Python, spaCy, NLTK, expresiones regulares y análisis gramatical, pero sobre todo aplicas criterio: sabes qué se puede eliminar, qué se debe conservar y qué debe revisarse con cuidado.

Ese es el aprendizaje central de esta unidad: el preprocesamiento no es una tarea mecánica previa al verdadero análisis. Es ya análisis lingüístico aplicado. Cada paso del pipeline decide qué versión del texto verá la máquina y, por tanto, qué podrá aprender de él.

Referencias

- Chu, C. y Wang, R. (2018). A survey of domain adaptation for neural machine translation. En *Proceedings of the 27th International Conference on Computational Linguistics* (pp. 1304–1319). Association for Computational Linguistics.
- de Marneffe, M.-C., Manning, C. D., Nivre, J. y Zeman, D. (2021). Universal Dependencies. *Computational Linguistics*, 47(2), 255–308. https://doi.org/10.1162/coli_a_00402
- González-Agirre, A., Marimon, M., Intxaurre, A., Rabal, O., Villegas, M. y Krallinger, M. (2019). PharmaCoNER: Pharmaceutical substances, compounds and proteins named entity recognition track. En *Proceedings of the 5th Workshop on BioNLP Open Shared Tasks*. Association for Computational Linguistics.
- Kudo, T. y Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics.
- Lima López, S., Pérez, N., Cuadros, M. y Rigau, G. (2020). NUBes: A corpus of negation and uncertainty in Spanish clinical texts. En *Proceedings of the Twelfth Language Resources and Evaluation Conference* (pp. 5772–5781). European Language Resources Association.
- Neumann, M., King, D., Beltagy, I. y Ammar, W. (2019). ScispaCy: Fast and robust models for biomedical natural language processing. En *Proceedings of the 18th BioNLP Workshop and Shared Task*. Association for Computational Linguistics.

- Petrov, S., Das, D. y McDonald, R. (2012). A universal part-of-speech tagset. En Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12). European Language Resources Association.
- Schmidt, C. W., Reddy, V., Zhang, H., Alameddine, A., Uzan, O., Pinter, Y. y Tanner, C. (2024). Tokenization is more than compression. En Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (pp. 678–702). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-main.40>
- Semenov, K., Huang, X., Zouhar, V., Berger, N., Zhu, D., Oncevay, A. y Chen, P. (2025). Findings of the WMT25 terminology translation task: Terminology is useful especially for good MTs. En Proceedings of the Tenth Conference on Machine Translation. Association for Computational Linguistics.
- Sennrich, R., Haddow, B. y Birch, A. (2016). Neural machine translation of rare words with subword units. En Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 1715–1725). Association for Computational Linguistics. <https://doi.org/10.18653/v1/P16-1162>
- Toporkov, O. y Agerri, R. (2024). On the role of morphological information for contextual lemmatization. Computational Linguistics, 50(1), 157–191. https://doi.org/10.1162/coli_a_00497
- Bird, S., Klein, E. y Loper, E. (2009). Natural language processing with Python: Analyzing text with the Natural Language Toolkit. O'Reilly Media.
- Loper, E. y Bird, S. (2002). NLTK: The Natural Language Toolkit. Proceedings of the ACL Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics (pp. 63-70). Association for Computational Linguistics. <https://aclanthology.org/W02-0109/>
- Manning, C. D., Raghavan, P. y Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press.
- Natural Language Toolkit Project. (2026). Natural Language Toolkit. NLTK. <https://www.nltk.org/>
- The pandas development team. (2026a). 10 minutes to pandas. pandas documentation. https://pandas.pydata.org/docs/user_guide/10min.html
- The pandas development team. (2026b). pandas.DataFrame. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
- The pandas development team. (2026c). pandas.DataFrame.to_csv. pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
- Wickham, H. (2014). Tidy data. Journal of Statistical Software, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>